CSE 221: Algorithms

Chapter 9 — Dynamic Programming

Prepared by Ariyan Hossain [AYO]

Dynamic Programming (DP), unlike Greedy Algorithm, is a method that solves all the subproblems and chooses the best one which gives the global optimal solution.

Dynamic programming, like divide and conquer, solves problems by combining solutions to subproblems. However, it improves efficiency by **storing results of subproblems** so each is solved only once. Instead of recomputing them, the algorithm reuses stored values, trading extra memory for faster performance and often reducing time complexity from exponential to polynomial.

Key idea: Solve each subproblem **once** and store the result to avoid recomputation.

When DP is Applicable

- **Optimal Substructure:** If the optimal solution of the whole problem can be built from the optimal solutions of its subproblems.
- **Overlapping Subproblems:** If the problem can be broken down into subproblems that are reused multiple times.
- It is especially common in **optimization problems** where we want a minimum or maximum value.

There are two standard implementation styles:

- **Top-down with memoization:** start from the top (original problem) and solve recursively, but save each computed subproblem so it is not solved again.
- **Bottom-up:** start from the smallest subproblems (base case) and build up to the larger problem (final answer) through filling a table.

Both approaches usually have the same asymptotic running time, though bottom-up often has smaller constant factors because it avoids recursive-call overhead.

1 Fibonacci numbers

The Fibonacci sequence is

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21,\ 34,\ 55,\ 89,\ \ldots$$

with the recurrence

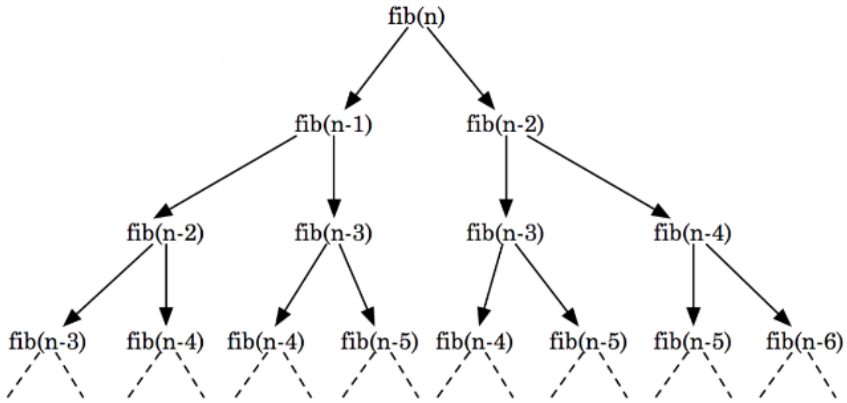
$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2), \quad \text{fib}(0) = 0,\ \text{fib}(1) = 1.$$

1.1 Recursive algorithm (top-down without memoization)

Recursive Fibonacci

```
FIBONACCI(n)
    if n = 0 or n = 1
        then return n
    else return FIBONACCI(n - 1) + FIBONACCI(n - 2)
```

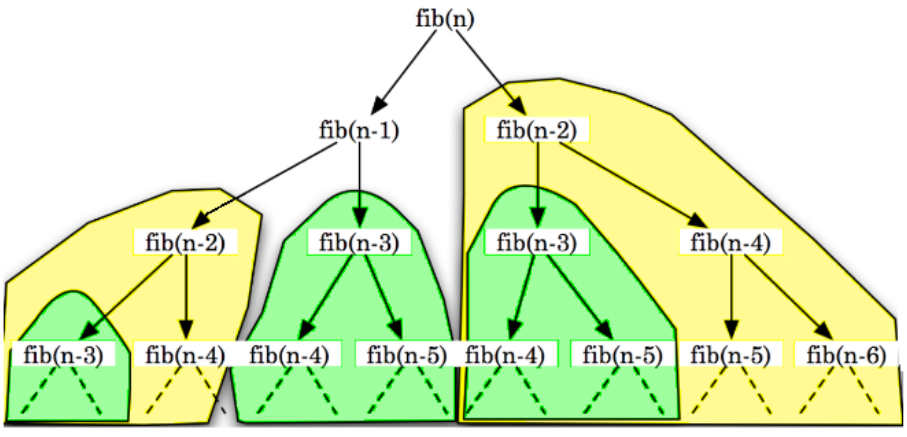
Recursive Tree:



Recursive tree of fib(n)

Time Complexity: The total number of nodes is $2^0+2^1+2^2+\cdots+2^n = \mathcal{O}(2^n)$, so $T(n) = \mathcal{O}(2^n)$.

The problem with the naive recursion is repetition. The same smaller Fibonacci values are recomputed many times.



Redundant computations of fib($n-2$) and fib($n-3$)

1.2 Memoization

Memoization means saving the answer of a subproblem the first time it is computed and reusing it later when needed again.

Memoization idea

At every recursive call:

1. check whether the subproblem has already been solved,
2. if yes, return the stored answer,
3. otherwise compute it, store it, and then return it.

For memoization, an additional data structure, such as an 1D or 2D array, is required to store the solutions of the subproblems.

Memoized Recursive Fibonacci

```
global F = [0..n]

FIBONACCI(n)
    if n = 0 or n = 1
        then return n
    if F[n] is empty
        then F[n] = FIBONACCI(n - 1) + FIBONACCI(n - 2)
    return F[n]
```

Time Complexity: Every value from $F[0]$ to $F[n]$ is filled only once in $\mathcal{O}(1)$, so $T(n) = \mathcal{O}(n)$.

1.3 Bottom-up Dynamic Programming

Instead of starting from n and going downward recursively, we can start from the base cases and build upward.

Bottom-up Fibonacci

```
global F = [0..n]

FIBONACCI(n)
    F[0] ← 0
    F[1] ← 1
    for i ← 2 to n
        do F[i] ← F[i - 1] + F[i - 2]
    return F[n]
```

Time Complexity: Every value from $F[0]$ to $F[n]$ is filled only once in $\mathcal{O}(1)$, so $T(n) = \mathcal{O}(n)$.

2 Knapsack Problem

Input: N items, where each item has a weight and a value (or profit), and a bag with capacity W.
Output: the set of items to include in the bag so that the total value is as large as possible, without exceeding the bag's weight capacity.

2.1 Fractional Knapsack

In fractional knapsack, an item can be taken partially. If an item weighs 2 kg, we may take only 0.5 kg if necessary.

Suppose the bag capacity is $\mathbf{W = 5\ kg}$ and the items are as follows.

Item	Value	Weight	Value index $= \frac{\text{value}}{\text{weight}}$
A	Tk. 140	2 kg	70
B	Tk. 200	1 kg	200
C	Tk. 150	5 kg	30
D	Tk. 240	3 kg	80

Since the goal is to maximize the total value while keeping the total weight as small as possible, a greedy approach is to compute the value-to-weight ratio for each item and then keep taking the items with the highest ratio until the bag reaches its capacity.

Item	Value	Weight	Value index $= \frac{\text{value}}{\text{weight}}$
B	Tk. 200	1 kg	200
D	Tk. 240	3 kg	80
A	Tk. 140	2 kg	70
C	Tk. 150	5 kg	30

Items sorted by value index (value per unit weight)

Step	Picked Item	Amount Taken	Remaining Capacity	Value Added
1	B	1 kg	4 kg	Tk. 200
2	D	3 kg	1 kg	Tk. 240
3	A	1 kg	0 kg	Tk. 70

So the total value is

$$200 + 240 + 70 = 510.$$

This is why a greedy approach works for fractional knapsack.

2.2 0/1 Knapsack

In 0/1 knapsack, each item is indivisible. An item must either be taken as a whole or not taken at all.

The following example shows why the greedy strategy is not always correct.

Item	Value	Weight	Value index
A	Tk. 300	3 kg	100
B	Tk. 190	2 kg	95
C	Tk. 180	2 kg	90

For capacity $W = 4$:

- Greedy by value index picks item A only, giving benefit 300.
- The optimal answer is items B and C, giving benefit $190 + 180 = 370$.

Therefore, greedy does not solve 0/1 knapsack in general.

2.3 Recursive algorithm of 0/1 knapsack

For each item, we make a choice: include it or exclude it.

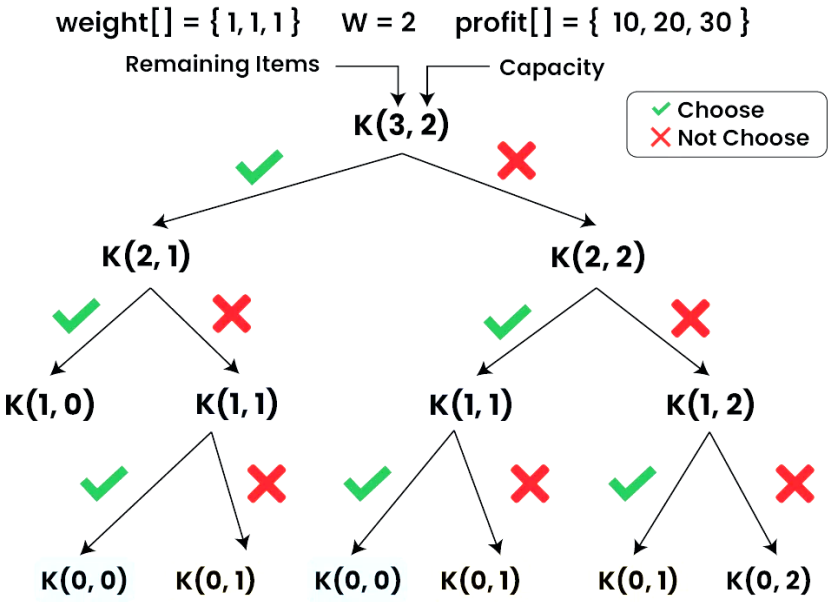
- If the current item's weight (w_i) is less than or equal to capacity (W), two choices are possible:
  - Include the item: add value (v_i), **or**
  - Exclude the item
  - We choose the option that gives the maximum profit.
  - In both cases, the number of remaining items becomes $(n - 1)$.
- If the current item's weight (w_i) is greater than capacity (W), once choices is possible:
  - Exclude the item
  - The number of remaining items becomes $(n - 1)$.
- Base case:
  - No items left ($n = 0$), **or**
  - Capacity becomes zero ($W = 0$)
  - In both cases, the result or value is 0.

Recursive 0/1 Knapsack

```
KNAPSACK(n, W)
    if n = 0 or W = 0
        then return 0
    elif w[n] > W
        then return KNAPSACK(n - 1, W)
    else return MAX(v[n] + KNAPSACK(n - 1, W - w[n]),
                    KNAPSACK(n - 1, W))
```

Here, $w[n]$ is the weight of the current item and $v[n]$ is its value.

Recursive Tree:



Recursive tree of K(3, 2)

Time Complexity: The total number of nodes is $2^0+2^1+2^2+\cdots+2^n = \mathcal{O}(2^n)$, so $T(n) = \mathcal{O}(2^n)$.

2.4 Memoized 0/1 Knapsack

Memoized Recursive 0/1 Knapsack

```
global M[0..n][0..W]

KNAPSACK(n, W)
    if n = 0 or W = 0
        then return 0
    elif M[n, W] is empty
        then if w[n] > W
                 M[n, W] ← KNAPSACK(n - 1, W)
             else M[n, W] ← MAX(v[n] + KNAPSACK(n - 1, W - w[n]),
                                KNAPSACK(n - 1, W))
    return M[n, W]
```

Time Complexity: Every value from $M[0][0]$ to $M[n+1][W+1]$ is filled only once in $\mathcal{O}(1)$, so $T(n) = \mathcal{O}(nW)$.

2.5 Bottom-up Dynamic Programming for 0/1 Knapsack

A 2D array (or a table) is built where the rows represent the first i items and the columns represent capacities from 0 to W.

Bottom-up 0/1 Knapsack

```
KNAPSACK(n, W)
    for i ← 0 to n
        do M[i, 0] ← 0
    for w ← 0 to W
        do M[0, w] ← 0
    for i ← 1 to n
        do for w ← 1 to W
               do if w[i] > w
                      then M[i, w] ← M[i - 1, w]
                   else M[i, w] ← MAX(v[i] + M[i - 1, w - w[i]],
                                      M[i - 1, w])
    return M[n, W]
```

Worked example

Given,

$$W = 8$$

Item	x_1	x_2	x_3	x_4
w_i	2	3	5	4
v_i	1	2	6	5

Initial DP Table with size $(n+1) \times (W+1)$:

	0	1	2	3	4	5	6	7	8
0									
1									
2									
3									
4									

To fill up the DP table, the items are ffirst sorted in ascending order according to their weights. After sorting, the processing order becomes

$$x_1,\ x_2,\ x_4,\ x_3.$$

Interpretation rule:

- If $w_i > w$, take the value of the upper cell
- Otherwise, take max(v_i + value of the upper row's $w - w_i$'s cell, value of the upper cell)

Final DP Table:

i	v_i	w_i
x_1	1	2
x_2	2	3
x_4	5	4
x_3	6	5

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1
2	0	0	1	2	2	3	3	3	3
3	0	0	1	2	5	5	6	7	7
4	0	0	1	2	5	6	6	7	8

Time Complexity: Every cell from $M[0][0]$ to $M[n+1][W+1]$ is filled only once in $\mathcal{O}(1)$, so $T(n) = \mathcal{O}(nW)$.

Backtracking the chosen items

Starting from $M[n, W] = M[4, 8] = 8$:

Row	**Remaining profit**	Decision
4th row	**8**	8 is not in the previous row, so include x_3 and reduce remaining profit to 2.
3rd row	**2**	2 already appears in the previous row, so do not include x_4.
2nd row	**2**	2 is not in the previous row, so include x_2 and reduce remaining profit to 0.
1st row	**0**	Nothing more needs to be included.

Thus the selected items are

$$x_1(0)\ x_2(1)\ x_3(1)\ x_4(0),$$

with total weight **8** and total profit **8**.

3 Longest Common Subsequence (LCS)

The longest common subsequence problem asks for the longest sequence that appears in two given sequences in the same relative order, but not necessarily contiguously.

Example:

- String 1: `abcdefghij`
- String 2: `cadgi`

The LCS is `cdgi` as it is the longest sequence that appears in both sequences in the same order.
.

3.1 Recursive algorithm of LCS

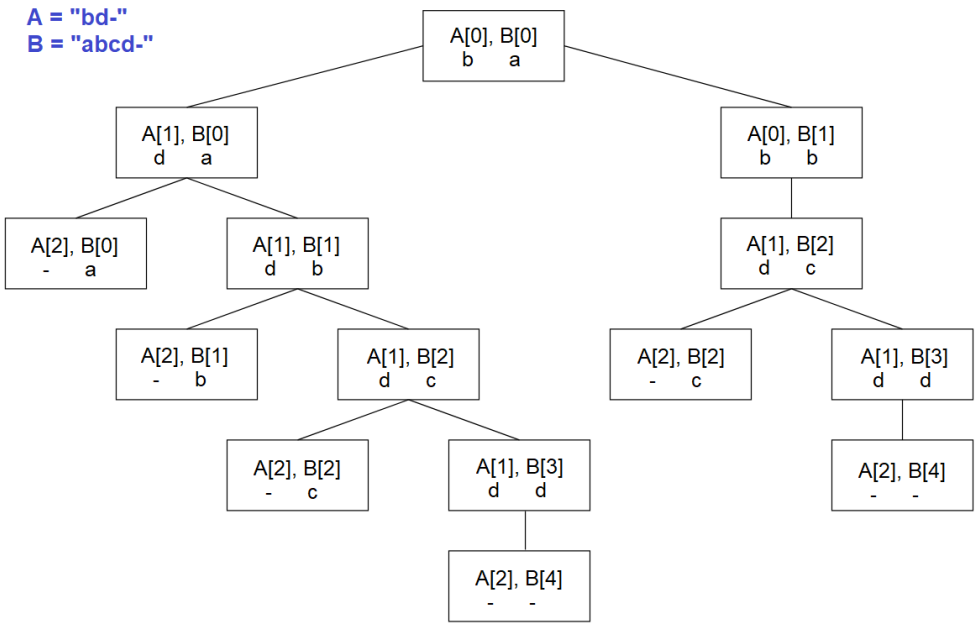
For each pair of characters, we make a choice: include the matching character or skip one character.

- If the current characters match, one choice is possible:
  - Include the character: add 1
  - Move to the next character in both sequences: $(i+1, j+1)$
- If the current characters do not match, two choices are possible:
  - Skip the current character of sequence A: $(i+1, j)$, **or**
  - Skip the current character of sequence B: $(i, j+1)$
  - We choose the option that gives the maximum LCS length.
- Base case:
  - No character left in sequence A (i), **or**
  - No character left in sequence B
  - In both cases, the result is 0.

Recursive LCS

```
LCS(i, j)
    if A[i] == "-" or B[j] == "-"
        then return 0
    elif A[i] == B[j]
        then return 1 + LCS(i + 1, j + 1)
    else return MAX(LCS(i + 1, j),
                    LCS(i, j + 1))
```

Recursive Tree:



Time Complexity: The total number of nodes is $2^0+2^1+2^2+\cdots+2^n = \mathcal{O}(2^n)$, so $T(n) = \mathcal{O}(2^n)$.

3.2 Memoized LCS

Memoized recursive LCS

```
global M[0..n][0..m]

LCS(i, j)
    if A[i] == "-" or B[j] == "-"
        then return 0
    elif M[i, j] is empty
        then if A[i] == B[j]
                  M[n, m] ← 1 + LCS(i + 1, j + 1)
             else M[n, m] ← MAX(LCS(i + 1, j),
                               LCS(i, j + 1))
    return M[i, j]
```

Time Complexity: Every value from $M[0][0]$ to $M[n+1][m+1]$ is filled only once in $\mathcal{O}(1)$, so $T(n) = \mathcal{O}(nm)$.

3.3 Bottom-up dynamic programming for LCS

A 2D array (or a table) is built where the rows represent the characters of sequence A and the columns represent the characters of sequence B.

Bottom-up LCS

```
global M[0..n][0..m]

LCS(n, m)
    for i ← 0 to n
        do M[i, 0] ← 0
    for j ← 0 to m
        do M[0, j] ← 0
    for i ← 1 to n
        do for j ← 1 to m
               do if A[i] == B[j]
                      then M[i, j] ← 1 + M[i - 1, j - 1]
                  else M[i, j] ← MAX(M[i - 1, j], M[i, j - 1])
    return M[n, m]
```

Worked example

Given,

$$A = \texttt{longest}, \qquad B = \texttt{stone}.$$

Initial DP Table with size $(n+1) \times (m+1)$:

	0	1	2	3	4	5	6	7
0								
1								
2								
3								
4								
5								

Interpretation rule:

- If characters match, take 1 + value of the leftmost diagonal cell
- Otherwise, take max(value of the left cell, value of the upper cell)

Final DP Table:

			l	o	n	g	e	s	t
		0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
s	1	0	0	0	0	0	0	1	1
t	2	0	0	0	0	0	0	1	2
o	3	0	0	1	1	1	1	1	2
n	4	0	0	1	2	2	2	2	2
e	5	0	0	1	2	2	3	3	3

Time Complexity: Every cell from $M[0][0]$ to $M[n+1][m+1]$ is filled only once in $\mathcal{O}(1)$, so $T(n) = \mathcal{O}(nm)$.

Backtracking the subsequence

Starting from $M[n, m] = M[5, 7] = 3$:

Column	**i**	**j**	**Current value**	Decision
7th	**e**	**t**	**3**	No match and 3 is in left cell, so move left.
6th	**e**	**s**	**3**	No match and 3 is in left cell, so move left.
5th	**e**	**e**	**3**	Match and 3-1 is in top-left diagonal, so **include e** and move top-left.
4th	**n**	**g**	**2**	No match and 2 is in left cell, so move left.
3rd	**n**	**n**	**2**	Match and 2-1 is in top-left diagonal, so **include n** and move top-left.
2nd	**o**	**o**	**1**	Match and 1-1 is in top-left diagonal, so **include o** and move top-left.
1st	**t**	**l**	**0**	No match and reached base case.

Thus the longest common subsequence is `one` with total length **3**.